

Analysis: ASeDatAb

Test Set 1

Whatever we try to do, the randomly-rotating judge in this test set might scuttle our plans. But we can fight randomness with randomness!

First, we can observe that if the judge ever tells us that the record has eight 1 bits, we have all but won (as long as we have at least one interaction left). This is because we can then submit 11111111, and no matter how the judge rotates it, the final result of XORing will be 00000000. Because of this, if we are ever told that the record has more than four 1s, it should be to our advantage to aim for eight 1s rather than zero 1s!

For now let's suppose the judge tells us there are b bits in the record, for some b between 1 and 4. (We will handle the other cases by symmetry, as explained above.) Let's do the following: choose a string with b bits uniformly at random, then send that. Now it doesn't really matter how the judge rotates our string -- whatever the resulting rotated string is, we were just as likely to pick that in the first place.

What are the possible outcomes? Suppose that $b = 2$. Then we have a $\frac{1}{\binom{8}{2}} = \frac{1}{28}$ chance of flipping both of the two 1s

(and therefore winning!), a $\frac{\binom{2}{1}\binom{6}{1}}{\binom{8}{2}} = \frac{12}{28}$ chance of flipping one of the two 1s and some innocent 0 (and thus being back

in the same $b = 2$ boat), and a $\frac{\binom{2}{0}\binom{6}{2}}{\binom{8}{2}} = \frac{15}{28}$ chance of missing both 1s and creating two new 1s (taking us to $b = 4$).

This may not seem too promising so far, but hang on...

Doing the same sort of analysis for the state $b = 4$, we find that we end up at one of $b = 0, 2, 4, 6, 8$, with probabilities $\frac{1}{70}, \frac{16}{70}, \frac{36}{70}, \frac{16}{70}, \frac{1}{70}$, respectively. But, as we mentioned, being at $b = 6$ is just the same as being at $b = 2$. If we are at $b = 6$, we can try to use two 1s to flip the two 0s, in the hopes of reaching 11111111. Similarly, being at $b = 8$ is (essentially) as good as being at $b = 0$. So we will lump those two probabilities into $b = 2$ and $b = 0$, respectively, getting transition probabilities to $b = 0, 2, 4$ of $\frac{1}{35}, \frac{16}{35},$ and $\frac{18}{35}$, respectively.

Notice that if b is even, we are trapped in the even- b -verse, randomly walking (according to those transition probabilities) until we either reach $b = 0$ or reach $b = 8$ or we run out of guesses. And if we are not in the even- b -verse, one round of sending exactly $\min(b, 8 - b)$ randomly placed 1s will get us there; we leave this as an exercise. So our strategy can be to spend up to two rounds getting into the even- b -verse, up to 297 rounds wandering, and up to one round possibly turning a 11111111 into a 00000000.

What are the chances that we will succeed in those 297 rounds? Observe that until we reach a winning state (00000000 or 11111111), we are always in one of two other states: $b = 2$ (lumped in with $b = 6$), or $b = 4$. In the former case, we have a $\frac{1}{28}$ chance of transitioning to a winning state, and in the latter case, that chance is $\frac{1}{35}$. Just for ease of argument, let's pessimistically guess that we get stuck hanging around in the less advantageous $b = 4$ state. But then to not win, we would still have to fail our $\frac{1}{35}$ lottery 297 times. The probability of that is $(1 - \frac{1}{35})^{297} \approx 0.0002$. So we have at least a 99.98% chance of succeeding with this strategy, and that is an overly conservative lower bound!

(If you're curious about the actual success probability, we can conservatively assume that we always start our journey

at $b = 4$, and then find the upper left cell of $\begin{pmatrix} 1 & \frac{1}{28} & \frac{1}{35} \\ 0 & \frac{12}{28} & \frac{16}{35} \\ 0 & \frac{15}{28} & \frac{18}{35} \end{pmatrix}^{297} \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$, which turns out to be about

$0.99993 \approx 99.993\%$).

Of course, we still have to pass *all* 100 test cases in Test Set 1, and the probability of this is a bit smaller: $\approx 0.99993^{100} \approx 0.993$. But 99.3% isn't so bad, and if we see an unlucky failure with this method, we can easily get another independent try by changing our code's random seed, since we have control over that source of randomness.

Test Set 2

In Test Set 2, the judge does not behave randomly and can and will choose rotation values that keep us away from reaching our goal. So, we need a strategy that is guaranteed to reset the record to all zeroes.

One way to do this is to consider the current *state*. Let's define a state as the set of all possible values the record could currently be set to. A key observation here is that two values that are cyclic rotations of each other are equivalent.

Therefore, we can eliminate these duplicates from our sets. After the first exchange, we know how many bits are set to 1 in the record. Thus, all potential states only have values that have the same bit count.

We can enumerate all possible values for each bit count (while removing duplicates that are cyclic rotations of another value). If we do this, we find the following sets of potential values:

- 0 bits: {00000000}
- 1 bit: {00000001}
- 2 bits: {00000011, 00000101, 00001001, 00010001}
- 3 bits: {00000111, 00001011, 00001101, 00010011, 00010101, 00011001, 00100101}
- 4 bits: {00001111, 00010111, 00011011, 00011101, 00100111, 00101011, 00101101, 00110011, 00110101, 01010101}
- 5 bits: {00011111, 00101111, 00110111, 00111011, 00111101, 01010111, 01011011}
- 6 bits: {00111111, 01011111, 01101111, 01110111}
- 7 bits: {01111111}
- 8 bits: {11111111}

Notice that the largest of these sets (4 bits) has only 10 elements. Thus, there are at most $2^{10} = 1024$ unique states for when we have 4 bits on. This is small enough to consider all possible states and so something similar to a BFS (breadth-first search) from the state with all zeroes.

Specifically, we can consider the set of all states that we know can be forced to reach all zeroes (initially just the solved state where the record is 00000000). Then, for a given state, A , we can consider trying all possible values for V and simulate the results of the 8 different rotation values the judge could choose. Grouping those by their bitcounts gives us the possible states that A could transition to for a specific value of V . If all of those states are ones we have processed, then we know that when state A , we can use this value of V to get us closer to setting the record to all zeroes.

It turns out that if we keep repeating the above process, we will eventually process all possible states. This gives us instructions on which numbers to provide for V given the current state. The alternative solution is proof provided below works to prove that this is always possible for 8-bit records.

Alternative Solution

It turns out that we can solve this problem without ever knowing the bit count after interactions (other than being told when we eventually reach 00000000).

Let's consider how we would solve this problem for a record that has only 1 bit. Since we know the value starts as not 0, we can force the state to reach 0 by sending 1 to the judge. Let's call this sequence $P[0]$:

1

Now, let's consider a record that is 2 bits (and is not all zeroes). Let's start by assuming that the two bits are the same. If that's the case, we can force the record to be all zeroes by sending 11. If we haven't reached all zeroes after that, that means our initial assumption that the left and right bit were the same was incorrect. So, if we send 10, we can make the two bits the same. Then, if we are still not all zeroes, we can send another 11. Let's call this sequence $P[1]$:

```
11    // P[0] + P[0]
10    // P[0] + 0
11    // P[0] + P[0]
```

Now, let's generalize this and assume that the record has 2^k bits and is not all zeroes. Let's assume that the left 2^{k-1} bits and the right 2^{k-1} bits are the same. If that's the case, we can use $P[k-1]$ (but each step is appended to itself) to force the record to reach all zeroes. If we did not reach all zeroes then our assumption that the left half and right were the same was not correct.

So, we can use the first instruction in $P[k-1]$ and append 2^{k-1} 0's to it. Then we can repeat all of $P[k-1]$ (each step doubled like before) again. As long as we keep not reaching all zeroes, we repeat this process with the next instruction in $P[k-1]$.

The following Python code shows this process for how we can generate $P[3]$ for 8-bit records:

```
def appendzero(s):
    return s + '0' * len(s)

def expand(s):
    return s + s

def P(k):
```

```
if k == 0:
    return ['1']
seq = P(k - 1)
seq_with_zero = [appendzero(s) for s in seq]
seq_with_copy = [expand(s) for s in seq]
res = seq_with_copy[:]
for ins in seq_with_zero:
    res += [ins]
    res += seq_with_copy
return res

print(P(3))
```

Analysis: Controlled Inflation

The key observation is that for each customer, either the increasing or decreasing order of target pressures will produce an optimal solution. That is, no other permutation of products will result in a strictly lower number of button presses. Let's prove this.

First, note that this is true for each individual customer i . At one point in the process, the pump will be at the minimal pressure $\text{Min}_i = \min_{j=1..P} X_{i,j}$. At another point, the pump will be at the maximal pressure $\text{Max}_i = \max_{j=1..P} X_{i,j}$. This means that we have to reach both of them, so we will always have to press the buttons at least $\text{Max}_i - \text{Min}_i$ times, no matter the order. This can be achieved by arranging the products in either increasing or decreasing order of their target pressures.

Now, the pressure also has to be adjusted between customers, which requires pressing the buttons. Let's see why all other product orders do not improve the answer by considering how many button presses between customers we can save. While processing the i -th customer, the pump will be at Min_i at one point, at Max_i at another point, and finally we will leave it at the pressure of the last product, $X_{i,\text{last}}$. This requires at least $(\text{Max}_i - \text{Min}_i) + (\text{Max}_i - X_{i,\text{last}})$ button presses, but the potential saving is at most $(\text{Max}_i - X_{i,\text{last}})$, which is the least amount of additional button presses we needed to do. If the pump reaches the maximum pressure before reaching the minimum pressure, a similar relationship holds, meaning this different order does not improve the answer.

Now that we know that it is enough to consider only the increasing and decreasing orders, we will only keep track of Min_i and Max_i for each customer. For the test set with the visible verdict, it is enough to check all $2^N \leq 1024$ possibilities of choosing the increasing or decreasing order for each customer and simulate the process.

To do this more efficiently, we can use dynamic programming. Let $dp_{i,0}$ be the answer after processing i customers where the products for the last customers are arranged in increasing order. Similarly, let $dp_{i,1}$ be the answer after the first i customers with the products for the last one arranged in decreasing order. We will also keep track of the pressure we left the pump at, l_0 and l_1 for the increasing and decreasing orders correspondingly. Clearly, $dp_{0,0} = dp_{0,1} = 0$ and $l_0 = l_1 = 0$ (the starting pressure). Now, assume dp is calculated up to i . The following equations give the values for the next customer:

$$dp_{i+1,0} = \min \left(\begin{array}{l} dp_{i,0} + |l_0 - \text{Min}_{i+1}| + (\text{Max}_{i+1} - \text{Min}_{i+1}), \\ dp_{i,1} + |l_1 - \text{Min}_{i+1}| + (\text{Max}_{i+1} - \text{Min}_{i+1}) \end{array} \right)$$

$$dp_{i+1,1} = \min \left(\begin{array}{l} dp_{i,0} + |l_0 - \text{Max}_{i+1}| + (\text{Max}_{i+1} - \text{Min}_{i+1}), \\ dp_{i,1} + |l_1 - \text{Max}_{i+1}| + (\text{Max}_{i+1} - \text{Min}_{i+1}) \end{array} \right)$$

And update the last pressures: $l_0 = \text{Max}_{i+1}$, $l_1 = \text{Min}_{i+1}$.

Analysis: Pancake Deque

Test Set 1

For the first test set, we could try a brute force solution for this problem. Given a remaining deque of pancakes $\mathbf{D}$, we could choose to serve the next customer from either the front or back of the deque. Meanwhile, we could update the maximum deliciousness of the pancakes served every time we decided to serve a pancake out. We could use a recursive brute force method to simulate the process.

Given that every time we have two choices for serving the pancake (either the first or the last one), the overall time complexity of the approach will be $O(2^N)$, which is sufficient to solve the first test set.

Notice that the brute force method can be improved by using memoization to record the current optimal solution for a partial deque. There are $\binom{N}{2} + N = \frac{N \cdot (N+1)}{2}$ partial deques in total to take into consideration. Therefore, this will reduce the time complexity down to $O(N^2)$.

Test Set 2

The same approach cannot be applied to the second test set, since it will result in a time limit exceed. Therefore we need a more clever way other than trying to serve the pancakes brute-force. We could quickly make an observation that, if the deliciousness of the pancakes we could serve at some point are $\mathbf{D}_{\text{left}}$ and $\mathbf{D}_{\text{right}}$, it will always be better to serve the one with less deliciousness. To prove this, we could do a quick analysis. For simplicity, let us assume that $\mathbf{D}_{\text{left}} \leq \mathbf{D}_{\text{right}}$, and denote $\mathbf{D}_{\text{max}}$ as the greatest deliciousness of the pancakes served out so far. If $\mathbf{D}_{\text{left}} < \mathbf{D}_{\text{max}}$, this means that the pancake will be served out free no matter when we serve it, so we could easily serve it out now and will not affect our final answer. Otherwise, since $\mathbf{D}_{\text{left}} \leq \mathbf{D}_{\text{right}}$ it will always be better to serve $\mathbf{D}_{\text{left}}$ first, or else $\mathbf{D}_{\text{max}}$ will be updated to at least $\mathbf{D}_{\text{right}}$. In that case $\mathbf{D}_{\text{left}}$ will be served out free. Therefore we could summarize into a criteria for serving pancakes: serve out $\min(\mathbf{D}_{\text{left}}, \mathbf{D}_{\text{right}})$, and update $\mathbf{D}_{\text{max}}$ if needed. For each customer, this criteria takes $O(1)$ time to perform, therefore the total time complexity of this approach is $O(N)$.